

Process Scheduling

In this project, you will simulate the process scheduling part of an operating system. You will implement a statistics-based scheduler, ignoring almost every other aspect of the OS. In this project we will be using message queues for synchronization.

In project 3, we had the children execute in rotation. In this project, instead the oss will be sending messages to child processes based on our statistics-based scheduler.

Please note that nothing in this project requires any sleep call or usleep call and any use of them indicates you are doing something wrong.

Operating System Simulator

This will be your main program and serve as the master process. You will start the operating system simulator (call the executable `oss`) as one main process who will fork multiple children at random times. The randomness will be simulated by a logical clock that will also be updated by `oss`.

In the beginning, `oss` will allocate shared memory for system data structures, including a process control table with a process control block for each user process. The process control block is a fixed size structure and contains information on managing the child process scheduling. Notice that since it is a simulator, you will not need to allocate space to save the context of child processes. But you must allocate space for scheduling-related items such as total CPU time used, total time in the system, your local simulated pid and the actual pid. The process control block resides in main memory and does not need to be accessible by the children. Since we are limiting ourselves to 20 processes in this class, you should allocate space for up to 18 process control blocks. Your process table should contain a method to keep track of what process control blocks are currently in use.

I suggest making your process table an array of structs of PCBs as below, but note you can create other fields. An example is below:

```
struct PCB {
    int occupied;           // either true or false
    pid_t pid;              // process id of this child
    int startSeconds;       // time when it was created
    int startNano;          // time when it was created
    int serviceTimeSeconds; // total seconds it has been "scheduled"
    int serviceTimeNano;    // total nanoseconds it has been "scheduled"
    int eventWaitSec;       // when does its event happen?
    int eventWaitNano;      // when does its event happen?
    int blocked;            // is this process waiting on event?
};

struct PCB processTable[20];
```

The OSS simulates time passing by using a simulated system clock. The clock is stored in two shared integers in memory, one which stores seconds and the other nanoseconds. will be in nanoseconds, so use two unsigned integers for the clock. `oss` will be creating user processes at random intervals. While the simulated clock (the two integers) are viewable by the child processes, it should only be advanced (changed) by `oss`.

`oss` will create user processes based on our inputs as in previous projects. However, the `-t` will be treated slightly differently.

Your solution will be invoked using the following command:

```
oss [-h] [-n proc] [-s simul] [-t timelimitForChildren]
    [-i fractionOfSecondToLaunchChildren] [-f logfile]
```

The `-t` parameter is the upper bound on how long the process should be "scheduled" before it decides it should terminate. It is an upper bound, so if `-t` is given a value of 2.5, launched processes should be given a random cpu burst from 0 to 2.5 seconds. `-i` is again the fraction of a second between launching children and can be 0. For example, if `-i` is given a value of 0.2, processes should not be generated faster than every 0.2 seconds.

`oss` acts as the scheduler and so will *schedule* a process by sending it a message using a message queue. If there are no processes currently ready to run in the system, it should increment the clock until it is the time where it should launch a process (for the situation where all children have already terminated but we aren't allowed to launch more immediately due to `-i`). It should then set up that process, generate a new time where it will create a new process and then using a message queue, schedule a process to run by sending it a message. It should then wait for a message back from that process that it has finished its task.

How our clock will operate The clock in this project is incremented differently. Essentially, `oss` now adds time to the clock to indicate something that it had to do. In this project `oss` simulates time passing in the system by adding time to the clock and as it is the only process that would change the clock. If a user process uses (uses in the sense was scheduled and so would be running on our cpu for a certain amount) some time, `oss` should indicate this by advancing the clock. In the same fashion, if `oss` does something that should take some time if it was a real operating system (for example, making a scheduling decision), it should increment the clock by a small amount to indicate the time it spent.

Only one process will be in the "running" state in our simulated system at a time as we are doing scheduling.

Note on incrementing the clock Note that you are not by default incrementing the clock. Instead, you are only incrementing it when work is done or if no processes are in the system. However, we do have to decide what our base time quantum is to schedule a process. For submission, I want a base time quantum of 10ms. You can of course use others for testing.

Scheduling Algorithm. Assuming you have more than one process in your simulated system, `oss` will *select* a process to run and *schedule* it for execution. It will select the process by using a scheduling algorithm using a statistics based scheduler. Every time `oss` has to make a scheduling decision it will calculate, for all non-blocked (ready) processes in the system, a ratio of total service time divided by their total time in the system. The process with the lowest ratio should be scheduled. If the denominator is zero, for this project you can treat this ratio as 0 (as it would mean the process hasn't had any chance to run at all yet). If there is a tie value, break the tie by scheduling the process with the lowest process id in your system.

Communication between processes

As described previously, the `oss` will be communicating with the user processes to "schedule" them. These messages should not simply be put into the queue for any to read, but instead sent from `oss` directly to a particular user process. When the user process is finished with its task, it will send a message back to `oss`, and only `oss`, that it is now done with its time quantum. The message sent should be at least (could be more if you want to add more) a number, indicating how much time quantum they will be scheduled for. The message back can also be an integer, positive to indicate how much time they used up, negative if they indicate they are terminating after using some time. For example, if `oss` sent 5000 to a user process, that would indicate it was being given a time quantum of 5000 ns. If it received a -2500 back, the `oss` would read that as the user process used 2500ns before it terminated. If it got a 2500 back, it would indicate it used 2500 before it had an i/o interrupt. If it got 5000 back, the `oss` would understand that to mean the process used its entire time quantum.

User Processes

All user processes simulate the system by performing some work that will take some random time. The user processes will wait on receiving a message giving them a time slice and then it will simulate running. They do not do any *actual* work but instead send a message to `oss` saying how much time they used and if they had to use i/o or had to terminate. To emphasize, processes indicate they have taken up some time by sending a message back, they do not do any "actual" running other than calculating the values below.

Note that processes should keep track of how much total time they have used up "running" as it is necessary to compare this to their total cpu burst time they were called with.

Processes have three things that they might need to do. One would be that they will terminate after using a percentage of the quantum they were assigned. Another would be that they are using their full time quantum. The other is that they are using part of their time quantum before having to do an i/o interrupt. This needs to be communicated back to `oss` and that used to determine how much time is used up in the system and what queue it will go into.

The choices are listed below for clarity:

- Process runs for its full time quantum. In this case, after the message is sent back, the clock would be incremented by the quantum it was scheduled for and the process would be put in one lower queue.
- Process runs for part of its time quantum and is interrupted. It would be put in the blocked queue and the clock would be incremented by how much time was used
- Process runs for part of its time quantum and terminates. The clock would be incremented by the amount of time used and the process would be cleared out of the system.

When processes were launched, they were given their total cpu burst. So the 3rd option would happen if the time they were scheduled is going to exceed the time they have to run in total. The process should send a message back to oss indicating that it terminated and how much of the time slice it was given it used "running" before it terminated.

For the other two options, you should have a probability stored as a constant that would indicate the percentage chance that a process, once scheduled, becomes blocked. Using that, every time a process is scheduled, it uses that probability to determinate if it uses up all the time running when it was scheduled or uses up a part of that time and then becomes blocked. Note that when creating this random number you must be careful that the seed you use is different for all processes, so I suggest seeding off of some function of a processes pid. If it is to become blocked, it should use a randomly determined amount of its timeslice and then send the amount it used and notification that it should become blocked for a bit.

If oss receives notification that a process is becoming blocked, it should calculate when it will become unblocked. To simplify this for this project, if a process becomes blocked, it should be blocked for 0.6 seconds. oss should store the time the process would become unblocked in its PCB of eventWaitSec and eventWaitNano. As you could have multiple processes blocked, it might be useful to store processes in a blocked queue or just sometimes iterate through the process table and see if some procsses should "wake" up. Note that the simulated work of moving a process from a blocked queue to a ready queue would take more time than a normal scheduling decision so it would make sense to increment the system clock to indicate this.

Make sure that you have signal handing to terminate all processes, if needed. In case of abnormal termination, make sure to remove shared memory and message queues.

Log Output

Your program should send enough output to a log file such that it is possible for me to determine its operation. For example:

```
OSS: Generating process with PID 3 and putting it in ready queue 0 at time 0:5000015
OSS: Ready queue priorities [0.05,0.07,0.80,...]
OSS: Dispatching process with PID 2 priority 0.05 from ready queue at time 0:5000805,
OSS: total time this dispatch was 790 nanoseconds
OSS: Receiving that process with PID 2 ran for 400000 nanoseconds
OSS: Putting process with PID 2 into ready queue
OSS: Ready queue priorities [0.05,0.07,0.80,...]
OSS: Dispatching process with PID 3 priority 0.12 from ready queue at time 0:5401805,
OSS: total time this dispatch was 1000 nanoseconds
OSS: Receiving that process with PID 3 ran for 270000 nanoseconds,
OSS: not using its entire time quantum
OSS: Putting process with PID 3 into blocked queue
OSS: Ready queue priorities [0.05,0.07,0.80,...]
OSS: Dispatching process with PID 1 from ready queue priority 0.11 at time 0:5402505,
OSS: total time spent in dispatch was 7000 nanoseconds
etc
```

Every half a second, output the process table.

Final report

Your simulation should end with a report on average wait time, average cpu utilization and average time a process waited in a blocked queue. Also include how long the CPU was idle with no ready processes.

I suggest not simply appending to previous logs, but start a new file each time. Also be careful about infinite loops that could generate excessively long log files. So for example, keep track of total lines in the log file and terminate writing to the log if it exceeds 10000 lines.

Note that the above log was using arbitrary numbers, so your times spent in dispatch could be quite different.

Overview of the procedure

```

while (stillChildrenToLaunch or childrenInSystem) {
    determine if we should launch a child
        if so, launch a child and update pcb

    check if any blocked process should be changed to ready
        if so, set it as unblocked

    if there is at least one ready process
        calculate priorities
        figure out which process has the lowest ratio
        schedule selected process by sending it a message
        receive a message back
        update appropriate structures
        update statistics like total wait time, etc
    else
        increment the clock by the appropriate amount
        so next iteration would launch a process

    Every half a second, output the process table and a list of blocked processes to the screen
}

```

You will want to increment your clock by a small amount everytime oss is updating one of its own data structures, as this is the work the oss is doing.

I highly suggest you do this project incrementally. A suggested way to break it down...

- Have master create a process control table with one user process and create that user process, testing the message queues back and forth.
- Schedule the one user process over and over, logging the data
- Start calculating priorities, add additional user processes, scheduling all user processes with it
- Add the chance for user processes to be blocked on an event, keep track of statistics on this.
- Keep track of and output statistics like throughput, wait time, etc

Do not try to do everything at once and be stuck with no idea what is failing.

Termination Criteria

oss should terminate if more than 3 real-life seconds have passed or if it has finished all the processes based on the -n parameter. As discussed previously, ensure that appropriate statistics are displayed. Keep in mind that if the percentage chance for a process to be blocked is high, processes might be blocked for almost all the time in the system, so make sure this percentage is fairly low. You should tune this so we see more than one process being unblocked in the system at least a bit.

Criteria for success

Make sure that you implement the specified scheduling algorithm and document it appropriately. *You must clean up after yourself.* That is, after the program terminates, whether normally or by force, there should be no shared memory, semaphore, or message queue that is left allocated to you.

Grading

1. *Overall submission: 10pts.* Program compiles and upon reading, seems to solve the assigned problem in the specified manner.
2. *README/Makefile: 10pts.* Ensure that they are present and work appropriately.
3. *Code readability: 10pts.* Code should be readable with appropriate comments. Author and date should be identified.
4. *Signal handling/cleaning up after yourself: 20pts.* The code terminates appropriately and all IPC primitives are cleaned up.
5. *Proper scheduling algorithm: 20pts.* The algorithm should be correct and appropriately documented.
6. *Conformance to specifications: 30pts.* Overall proper implementation and documentation.

README

It is required that you submit a README file. This file should start with the following:

```
Name: Yourname
Date: When project started
Environment: What environment you used (linux, vi)
How to compile the project:
    Type 'make'
Example of how to run the project:
    ./oss -n 3 -s 2 -t 4 -i 0.6 -f log.txt
```

In addition, if you use generative AI, you must include a section describing your use of generative AI in detail. This should include a bare minimum of what generative AI was used and some list of prompts that you used. I also want at least a few sentence description of in general how useful you found it.

If you are not using generative AI, please explain why not. This is not a problem of course.

For example, at a bare minimum something like:

Generative AI used : chatgpt

Prompts:

```
Please give me a C++ command line program to play tic-tac-toe between two people
Change the previous code to add the capability to play multiple games
Add the capability to keep track of the previous record of players.
Explain to me why dogs like fire hydrants.
Given the previous code, please add a command line option
to allow one player to cheat
etc
```

Summary: The initial game generated worked well, but it had one bug that I had to fix.

It did not work to generate the cheating option as chatgpt claimed I was unethical.

Deliverables

Handin an electronic copy of all the sources, README, Makefile(s), and results. Create your programs in a directory called *username.4* where *username* is your login name on hoare. Once you are done with everything, *remove the executables and object files*, and issue the following commands:

```
chmod 700 username.4
```

```
cp -p -r username.4 /home/hauschildm/cs4760/assignment4
```

Do not forget `Makefile` (with suffix rules), version control, and `README` for the assignment. If you do not use version control, you will lose 10 points. Omission of a `Makefile` (with suffix rules) will result in a loss of another 10 points, while `README` will cost you 5 points.